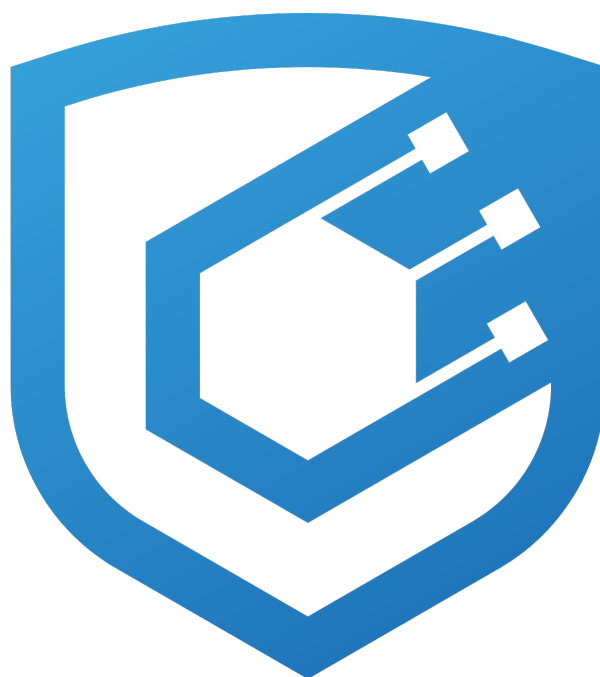




XAUM Sui Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[MoveJay](#)

[BlackPanther](#)

January 27, 2026

Contents

1	About Cyfrin	2
2	Disclaimer	2
3	Risk Classification	2
4	Protocol Summary	2
5	Audit Scope	2
6	Executive Summary	2
7	Findings	4
7.1	Low Risk	4
7.1.1	execute_mint_to, redeem, cc_receive never enforce the blocklist	4
7.2	Informational	5
7.2.1	Role table documentation mismatch for cc_send_mint_budget	5
7.2.2	Missing Zero Amount Validation in Cross-Chain Functions	5
7.2.3	Events Not Emitted for Key Administrative Operations	6
7.2.4	Unvalidated Refund Address Parameter in Cross-Chain Send Functions	6
7.2.5	Cross-Chain Token Transfers Bypass mintBudget Accounting	7
7.2.6	Legacy Move Syntax Patterns	8
7.2.7	Receiver Address Lacks Length Bounds Validation	8

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the solidity implementation of the contracts.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

Matrixdock is a Real World Asset (RWA) Tokenization protocol which has tokenized US Treasuries as STBT and physical gold as XAUm.

The focus of this audit is the LayerZero integration on Sui enabling cross-chain transfers between Sui and other chains such as EVM, Solana etc.

5 Audit Scope

The audit scope was limited to:

```
xaum-sui/packages/messenger_lz/sources/messenger_oapp.move
xaum-sui/packages/messenger_lz/sources/ptb_builder.move
xaum-sui/packages/mtoken/sources/mtoken.move
xaum-sui/packages/mtoken/sources/message_codec/message_writer.move
xaum-sui/packages/mtoken/sources/message_codec/message_reader.move
xaum-sui/packages/mtoken/sources/message_codec/message_codec.move
```

6 Executive Summary

Over the course of 4 days, the Cyfrin team conducted an audit on the [XAUM Sui](#) smart contracts provided by [Matrixdock](#). In this period, a total of 8 issues were found.

The findings consist of 1 Low severity issues with the remainder being informational.

Summary

Project Name	XAUM Sui
Repository	RWA-Contracts
Commit	f22eb309d173...
Fix Commit	fc0532d6384b...
Audit Timeline	Jan 19th - Jan 22nd, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	0
Low Risk	1
Informational	7
Gas Optimizations	0
Total Issues	8

Summary of Findings

[L-1] <code>execute_mint_to</code> , <code>redeem</code> , <code>cc_receive</code> never enforce the blocklist	Resolved
[I-1] Role table documentation mismatch for <code>cc_send_mint_budget</code>	Resolved
[I-2] Missing Zero Amount Validation in Cross-Chain Functions	Resolved
[I-3] Events Not Emitted for Key Administrative Operations	Acknowledged
[I-4] Unvalidated Refund Address Parameter in Cross-Chain Send Functions	Resolved
[I-5] Cross-Chain Token Transfers Bypass <code>mintBudget</code> Accounting	Acknowledged
[I-6] Legacy Move Syntax Patterns	Resolved
[I-7] Receiver Address Lacks Length Bounds Validation	Acknowledged

7 Findings

7.1 Low Risk

7.1.1 `execute_mint_to`, `redeem`, `cc_receive` never enforce the blocklist

Description: Sui's deny list v2 has an inherent timing gap: blocked addresses cannot send coins immediately (protocol enforced), but can still receive coins until the start of the next epoch (up to ~24 hours).

`mtoken.move` contains blocklist functionality however `execute_mint_to`, `redeem`, and `cc_receive` never explicitly check the blocklist, relying solely on Sui's protocol-level enforcement. This allows tokens to be sent to blocked addresses during the current epoch window.

For cross-chain operations via `cc_receive`, this creates additional risk: a message sent from another chain while the receiver was not blocked could arrive after the epoch boundary and fail, resulting in tokens burned on the source chain but never minted on the destination.

Recommended Mitigation: Consider the following:

- `execute_mint_to` should check if the recipient is blocked before minting
- `cc_receive` should check if the receiver is blocked before minting and transferring
- `redeem` should check if the customer is blocked

Use `coin::deny_list_v2_contains_next_epoch<T>(deny_list, addr)` to check blocklist status, which provides protection during the current epoch gap by identifying addresses that will be blocked.

Matrixdock: Fixed in commit [70fc8cc](#).

Cyfrin: Verified.

7.2 Informational

7.2.1 Role table documentation mismatch for `cc_send_mint_budget`

Description: The role permission table in [mtoken.move](#) documents that `mtoken::cc_send_mint_budget` only requires the Messenger role. However, the actual implementation requires both the Operator role AND a valid `MessengerCap`

```
public fun cc_send_mint_budget<T>(  
    state: &mut State<T>,  
    msg_cap: &MessengerCap,  
    amount: u64,  
    ctx: &mut TxContext,  
) : vector<u8> {  
    check_version(state);  
    check_operator(state, ctx); // requires operator role  
    check_messenger_cap(state, msg_cap); // requires valid MessengerCap  
    deduct_mint_budget(state, amount);  
    ...  
}
```

The documentation understates the actual access control requirements.

Impact: Inaccurate documentation may cause integration failures if developers assume only the Messenger role is required, when in reality the caller must also be the Operator.

Recommended Mitigation: Update the role permission table to accurately reflect the implementation

Ops\Roles\Delayed	Owner	Operator	Revoker	Messenger	Delayed
cc_send_mint_budget					

Matrixdock: Fixed in commit [e3f62f7](#).

Cyfrin: Verified.

7.2.2 Missing Zero Amount Validation in Cross-Chain Functions

Description: The functions `MToken::cc_send_token` and `MToken::cc_send_mint_budget` in the Sui implementation do not validate that the `amount` parameter is non-zero before processing cross-chain transfers. This allows users to initiate cross-chain messages with zero-value transfers, which will still incur LayerZero messaging fees despite transferring no actual value. The EVM implementation prevents this with the `_checkZeroValue` helper, which is called in both `ccSendToken` and `ccSendMintBudget`. This validation is absent from the corresponding Sui functions.

Impact:

1. Message Queue Pollution: Zero-value messages are still sent through LayerZero's messaging infrastructure, consuming queue capacity and network resources for meaningless transfers.
2. Event Log Spam: The `CCSendTokenEvent` and `CCSendMintBudgetEvent` events are emitted regardless of amount, polluting on-chain event logs with zero-value entries that provide no useful information.
3. Off-Chain Indexer Noise: Analytics systems, block explorers, and indexers tracking cross-chain activity would need to filter out these zero-value transfers, adding unnecessary complexity and noise to transaction histories.
4. Wasted User Fees: Users inadvertently sending zero amounts still pay LayerZero messaging fees for a transaction that accomplishes nothing.

Recommended Mitigation: Add zero amount validation to both functions to maintain parity with the EVM implementation and prevent wasteful transactions:

```
const EZeroValue: u64 = 13; // or appropriate error code

fun check_non_zero(amount: u64) {
    assert!(amount > 0, EZeroValue);
}
```

Then call `check_non_zero` with the `amount` parameter at the start of both `cc_send_token` and `cc_send_mint_budget`.

Matrixdock: Fixed in commit [26fd445](#).

Cyfrin: Verified.

7.2.3 Events Not Emitted for Key Administrative Operations

Description: Several administrative functions in the Sui `mtoken` module do not emit events, creating gaps in auditability and off-chain monitoring capabilities.

The `cc_new_messenger_cap` function creates or updates the messenger capability without emitting any event.

Additionally, the following revoke functions silently delete pending requests without emitting events:

- `revoke_transfer_ownership`
- `revoke_set_operator`
- `revoke_set_revoker`
- `revoke_set_delay`
- `revoke_mint_to`

Recommended Mitigation: Add events to these functions:

```
public struct SetMessengerCapEvent has copy, drop {
    old_cap_id: Option<ID>,
    new_cap_id: ID,
    holder: address,
}

public struct RequestRevokedEvent has copy, drop {
    req_id: ID,
    req_type: u8, // 1=TransferOwnership, 2=SetOperator, 3=SetRevoker, 4=SetDelay, 5=MintTo
    revoker: address,
}
```

Emit `SetMessengerCapEvent` in `cc_new_messenger_cap` and `RequestRevokedEvent` in each revoke function.

Matrixdock: Acknowledged. The team will add these events in future updates; for now, the current implementation will remain as is.

7.2.4 Unvalidated Refund Address Parameter in Cross-Chain Send Functions

Description: The `send_token` and `send_mint_budget` functions in `messenger_oapp.move` accept a `refund_address` parameter that specifies where excess LayerZero fees should be returned. This address is passed directly to the LayerZero endpoint without any validation:

```
public fun send_token(
    state: &State,
    mt_state: &mut MtState<XAUM>,
    my_oapp: &mut OApp,
    dst_eid: u32,
    extra_options: vector<u8>,
```

```

    native_token_fee: Coin<SUI>,
    refund_address: address,           // No validation performed
    receiver: vector<u8>,
    xaum_token: Coin<XAUM>,
    ctx: &mut TxContext,
): (Call<SendParam, MessagingReceipt>, SendContext) {
    //
    let lz_call = my_oapp.lz_send(
        &state.oapp_call_cap,
        dst_eid,
        msg_data,
        options,
        native_token_fee,
        option::none(),
        option::some(refund_address), // Passed without validation
        ctx,
    );

```

The following validations are missing:

- No check for zero address (@0x0)
- No check that refund address matches the transaction sender

In contrast, the EVM implementation always refunds excess fees to `msg.sender`:

```

if (msg.value - fee > 0) {
    payable(msg.sender).sendValue(msg.value - fee);
}

```

Recommended Mitigation: Always use `ctx.sender()` as the refund address to match the EVM implementation, or add a zero address check if keeping the parameter.}

Matrixdock: Fixed in commit [283381b](#).

Cyfrin: Verified.

7.2.5 Cross-Chain Token Transfers Bypass `mintBudget` Accounting

Description: Cross-chain token transfers do not properly update the `mint_budget` on either the source or destination chain, creating an accounting inconsistency.

When tokens are transferred cross-chain:

- **Source chain:** Tokens are burned but `mint_budget` is NOT increased
- **Destination chain:** Tokens are minted but `mint_budget` is NOT decreased or checked

This behavior differs from other minting/burning operations:

- `execute_mint_to` correctly deducts from `mint_budget` before minting
- `redeem` correctly adds to `mint_budget` after burning

As a result, a chain's `totalSupply` can exceed its `mint_budget` through incoming cross-chain transfers, and the source chain's `mint_budget` does not recover capacity when tokens leave.

Recommended Mitigation: If `mint_budget` is intended to track minting capacity per chain, cross-chain transfers should maintain this invariant:

1. **On send (`cc_send_token`):** Increase `mint_budget` when tokens are burned for cross-chain transfer, similar to how `redeem` handles burning
2. **On receive (`cc_receive` for token messages):** Decrease `mint_budget` when tokens are minted from incoming cross-chain transfers, similar to how `execute_mint_to` handles minting

Important consideration: If this mitigation is applied, cross-chain transfers may fail if the destination chain lacks sufficient `mint_budget`. The team should evaluate whether to:

- Pre-allocate `mint_budget` across chains to support expected cross-chain volume
- Implement a mechanism to transfer `mint_budget` alongside token transfers
- Accept the current design if `mint_budget` is only intended to control direct operator minting and not cross-chain flows

Matrixdock: Acknowledged; this is the expected behavior.

7.2.6 Legacy Move Syntax Patterns

Description: The codebase contains several code quality issues that deviate from Move 2024 conventions and best practices as documented in the [Move Book Code Quality Checklist](#). While these do not affect security or runtime costs, they impact code maintainability and readability.

Legacy Syntax Patterns

- **Dynamic field keys** use empty struct syntax (`struct Foo {}`) instead of positional syntax (`struct Foo();`)
- **Vector operations** use legacy module functions (`vector::length()`, `vector::empty()`) instead of struct methods (`bytes.length()`, `vector[]`)
- **While loops** used where iteration macros (`.fold!()`, `.do!()`) would be clearer
- **Struct initialization** does not use field shorthand (`sender: sender` instead of `sender`)

Recommended Mitigation: Update to Move 2024 syntax - Apply the following modernizations:

```
// Dynamic field keys
public struct TreasuryCapKey() has copy, drop, store;
public struct DenyCapKey() has copy, drop, store;
public struct MessengerCapKey() has copy, drop, store;

// Vector operations
let mut result = vector[];           // instead of vector::empty()
let len = bytes.length();           // instead of vector::length(bytes)
let b = bytes[i];                   // instead of *vector::borrow(bytes, i)
bytes.reverse();                     // instead of vector::reverse(&mut bytes)

// Loop macros
bytes.fold!(0u256, |acc, b| (acc << 8) + (*b as u256))

// Field shorthand
CCInboundToken { sender, receiver: address::from_bytes(receiver), }
```

References

- [Move Book: Code Quality Checklist](#)
- [Move 2024 Migration Guide](#)

Matrixdock: Fixed in commit [8d621ef](#).

Cyfrin: Verified.

7.2.7 Receiver Address Lacks Length Bounds Validation

Description: The `cc_send_token` function in `mtoken.move` and the `send_token` function in `messenger_oapp.move` accept a receiver parameter as `vector<u8>` without enforcing minimum or maximum length bounds. While the

`check_dst_addr` function validates that the receiver matches a configured `addr_len`, this check is bypassed when `addr_len` is set to zero, and there are no hardcoded bounds preventing extreme values.

The receiver parameter can be:

- **Zero-length:** An empty vector `b""` would pass validation when `addr_len = 0`
- **Arbitrarily long:** No maximum length prevents users from submitting excessively large receiver addresses, potentially causing:
 - Increased gas/storage costs on the destination chain
 - Message rejection by LayerZero or the destination chain
 - Wasted cross-chain fees for the user

Recommended Mitigation: Consider adding minimum and maximum length validation as defense-in-depth. This would involve:

1. Requiring non-zero `addr_len` when configuring destination chains via `set_peer`, ensuring all destination chains have explicit address format requirements.
2. Adding hardcoded bounds in `check_dst_addr` to reject empty addresses (minimum 1 byte) and excessively long addresses (maximum 64 bytes), providing a safety net regardless of per-chain configuration.

This protects users from self-inflicted errors where malformed receiver addresses would result in lost cross-chain fees without successful token delivery.

Matrixdock: Acknowledged. The address field was changed to accept arbitrary-length bytes to support non-EVM chains (Sui, Solana use 32 bytes) and chains with variable address lengths (e.g., Canton). For fixed-length address chains, `dst_addr_len` can be configured via `set_peer` to enforce validation. For variable-length chains, the check is intentionally skipped.